

Refactoring and Scaling scikit-bio's Distance Matrices

Personal Details

Name: Andrés Fernández Cervell

University: Universidad de Granada, Spain.

Email: andresfernandezcervell@gmail.com

Linkedin: Andrés Fernández Cervell | LinkedIn

Abstract

The increasing scale of biological datasets has rendered dense distance matrices computationally prohibitive. Modern bioinformatics requires scientific libraries to interoperate seamlessly with modern array frameworks and hardware accelerators. This 175-hour project aims to refactor scikit-bio's distance matrix infrastructure by implementing a new class hierarchy: `PairwiseMatrix` > `SymmetricMatrix` > `DistanceMatrix`, which will allow users to work with condensed, sparse and quantized matrices.

The new architecture will support the Python Array API standard (NEP 47) enabling interoperability with libraries such as Numpy, CuPy, PyTorch, JAX, Dask, and others. Furthermore, Spectral Computations will be designed using a `LinearOperator` for a partial eigen-decomposition and inverse calculations will be replaced by iterative Krylov Methods. This transition positions scikit-bio at the intersection of traditional bioinformatics and modern AI-driven discovery.

About me

I am a second year student currently pursuing a Double Degree in Mathematics and Computer Science. This background allows me to bridge the gap between abstract mathematical theory and high-performance software engineering. My experience includes:

- **Contributions to scikit-bio:** I have already successfully implemented a Proof of Concept refactoring alpha diversity metrics to support the Array API. This allowed me to gain a deeper comprehension of the standard, scikit-bio repository and communicative skills with the team.
 - scikit-bio array API refactor (Proof of Concept)
- **Algorithmic Research:** I recently contributed to the Gambit project, where I developed a mathematical report and C++ implementation using Sard's Theorem to resolve numerical stability issues in Nash equilibrium solvers using iterative methods.
 - Fix #492
 - Full Technical Report
- **Machine Learning and Big Data for Bioinformatics (MOOC):** Enrolled in a specialized course at UGR to apply Machine Learning fundamentals to real-world biological and genetic data.

- **Academic Background:** a current GPA of 8.0/10.0 provides me with a strong technical foundation in fields such as Linear Algebra, Algorithms and Data Structures. Experience with Python, C/C++, Java and Ruby. Award-winning communication skills (winner of Provincial Debate Tournament) and academic excellence, as well as fluency in Spanish (native), English (C1- Cambridge-Grade A) and French (B2-DELF).

Motivation, Impact & Technical Methodology

The Bottleneck in Modern Bioinformatics

Historically, pairwise relationships in biology (e.g., sequence distances, beta diversity) have been stored as dense matrices. However, as modern biological datasets scale, storing these relationships in dense $\mathcal{O}(n^2)$ structures becomes computationally prohibitive. A 100,000 sample dataset quickly exceeds the RAM limits of standard research hardware.

Currently, scikit-bio's `DistanceMatrix` provides an intuitive interface (a matrix combined with a sequence of IDs) for working with pairwise biological relationships. However, as the official documentation notes, distances are natively stored in a redundant, dense square-form format. While it offers basic utility methods to retrieve data in a condensed vector form, the underlying data structure fundamentally relies on dense, floating-point arrays, which leads to memory, time and hardware limitations.

Impact and Community Benefits

This project will resolve this bottleneck by redesigning scikit-bio's distance matrix infrastructure to be scalable, efficient, and ecosystem-friendly. By introducing sparse and condensed (vector-form) data structures, researchers will be able to process vastly larger datasets, with great applications in AI and Machine Learning.

Technical Methodology

The implementation will be modular, utilizing the `xp` namespace to ensure full Array API compatibility. The development is divided into the following blocks:

1. Core Architecture

Three-Class Hierarchy: The current two-class design (`DissimilarityMatrix` > `DistanceMatrix`) will be replaced with a three-class design: `PairwiseMatrix` > `SymmetricMatrix` > `DistanceMatrix`.

- `PairwiseMatrix`: The base class for nxn relationship matrices. A `device` attribute will be added so that exceptions are thrown when trying to operate with two matrices of a different device (CPU/GPU) at the same time.
- `SymmetricMatrix`: Enforces symmetry.
- `DistanceMatrix`: Enforces both symmetry and hollowness (diagonal fixed to 0).

To handle the state of each matrix, Boolean flags will be used, such as `sparse` (matrix that only stores non-zero data), `vector` (condensed in a 1D array) and `quantized` (the dense or sparse array contains only integer values to save memory). They will be read-only, generated automatically upon initialization or conversion. Furthermore, methods to check mathematics properties will be implemented, such as `is_symmetric` or `is_hollow`.

2. Data Representations

Condensed (Vector) vs. Square Forms: I will implement `to_square()` and `to_vector()` conversion methods after studying SciPy functions and the already implemented algorithm in scikit-bio. This transition between redundant 2D arrays and condensed 1D arrays will save up to $\sim 50\%$ memory.

Diagonal Handling: A `diagonal` attribute will be implemented to handle values independently from the upper-triangle vector (defaulting to 0 in `DistanceMatrix`), ensuring zero-copy operations and seamless reconstruction when converting back to square form.

Sparse Matrices & Graph Construction: Using SciPy's sparse module and CSR (Compressed Sparse Row), a graph data structure will be used to save memory, storing only non-zero data. I will introduce a `null_value` attribute (vital for differentiating between mathematical zeros and structural absences) and a thresholding logic to filter values upon initialization.

3. Spectral Layer & Laplacian Solvers

For massive graphs, materializing a dense matrix is computationally catastrophic. I will implement a Graph and Spectral Layer that avoids densification:

- **Partial Eigen-decomposition:** use of a `LinearOperator` and delegation to native solvers of Scipy. My approach of deterministic iterative solvers will complement the current work that is being done in scikit-bio for reaching approximate top-k subspaces.
- **Replacing Inverses with Krylov Solvers:** Explicit inverses on sparse matrices destroy memory optimization by densifying the output. I will implement `laplacian_solve()` using the iterative method Preconditioned Conjugate Gradient. Since Laplacians are singular on connected graphs by definition, to ensure well-posedness, the `laplacian_solve` method will implement explicit constraint strategies, such as `constraint="ridge"` (adding an epsilon to the diagonal), `constraint="ground_node"` (eliminating a row/column to remove the singularity), and `constraint="mean_zero"` (forcing the solution to be orthogonal to the all-ones vector, $x \perp 1$). A parameter `solver` will also be needed for calling the function, so that in the future, more iterative methods can be easily added to scikit-bio. The use of Numba to optimize the loops of the algorithm will be studied in the future. Furthermore, this design not only improves scalability but also aligns naturally with implicit differentiation strategies used in modern autodiff systems. This means that my operator will be designed to not preclude, theoretically, differentiable use cases (like gradients) for backends that support it, such as JAX or PyTorch. However, since this is not a priority, any remaining incompatibilities will be clearly documented to facilitate future improvements.

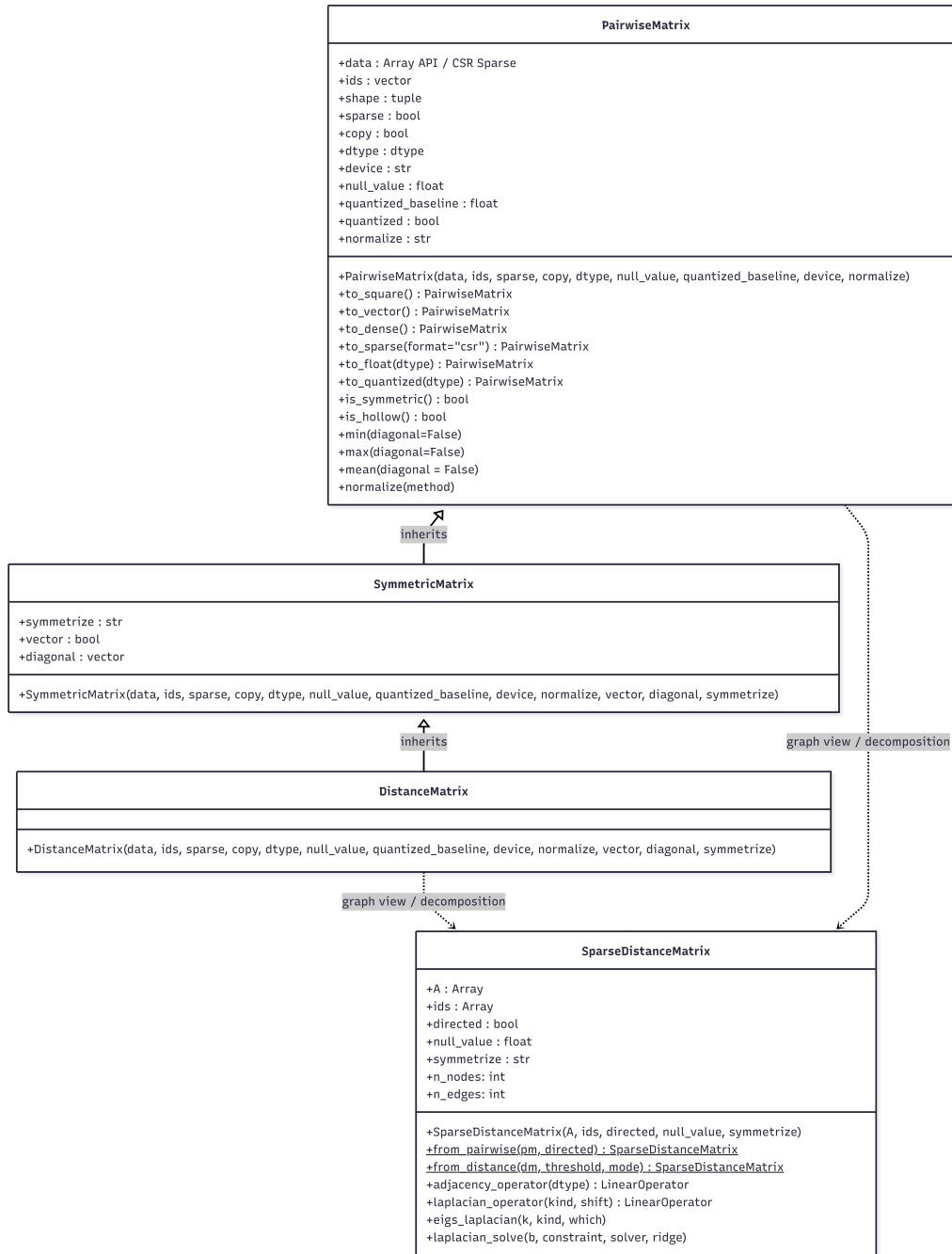
4. Testing & Performance Validation

Tests: To ensure the architectural integrity of the new hierarchy, I will implement a test suite using `pytest`, focusing on property-based testing (via Hypothesis) to verify mathematical invariants (symmetry, hollowness...) across all Array API.

Benchmarking: Scalability reports will be generated to monitor improvements in memory peak usage and execution time, to provide empirical evidence of the RAM and time savings.

5. Stretch Goals (Time Permitting)

Quantized Matrices: To drastically reduce memory, matrices will support integer scaling such as `int8`, `uint16` tied to a `quantized_baseline` float attribute. Methods like `to_quantized()` and `to_float()` will be implemented, including strict overflow protection to prevent data corruption. **Symmetrization & Normalization:** For asymmetric inputs, a `symmetrize` parameter will apply mathematical pooling (mean, max, min). Utility methods such as `max`, `min`, and `normalize` will be implemented with a `diagonal=False` parameter to avoid trivial zeros during calculations.



A provisional and simplified class diagram is shown above. It is structured to be realistic within the 175-hours project, and it is open to adapt to best serve the performance needs of the scikit-bio ecosystem. While it closely follows the requested API sketch, several deliberate architectural choices have been made:

- **Diagonal Management and Memory Optimization:** In the base `PairwiseMatrix` class, data is maintained in a 2D structure, meaning the diagonal is natively accessible at all times, making it redundant to add a `diagonal` attribute. However, when subclassing into `SymmetricMatrix` and `DistanceMatrix`, we introduce the ability to compress the matrix into a 1D vector that naturally discards the diagonal elements, so the explicit `diagonal` attribute becomes necessary. This ensures we can seamlessly reconstruct the square matrix without any data loss.
- **Explicit Boolean Attributes vs. Flag Mapping:** I have opted to represent the state

of the matrix using independent boolean attributes (e.g., `sparse`, `quantized`, `vector`, `copy`) rather than a single read-only dictionary mapping (`obj.flags`). I find it more intuitive, especially since `vector` is only useful for symmetric matrices. Giving each flag more independence makes it easier to understand and to work with. Nevertheless, I recognize the initial API sketch suggested a mapping approach; this is a design choice I am eager to discuss and align with the mentors.

Potential Challenges & Mitigations

- **Challenge: Array API Backend Incompatibilities:** Different backends may have subtle differences in type promotion or lack specific functions (for example, older versions may lack `count_nonzero`, added in version 2024.12)
- *Mitigation:* As discussed with my mentors, I will strictly adhere to the API standards NEP 47 Type Promotion Rules and use the `xp` namespace safely, assuming that all backends used will work on the latest update.
- **Challenge: Numerical Instability in Singular Laplacian:** Graph Laplacians are singular on connected graphs. Passing them to an iterative solver without care will cause divergence or unexpected behavior.
- *Mitigation:* I will use Krylov solvers (e.g., Preconditioned CG) combined with explicit constraint strategies (`mean_zero`, `ground_node`, or `ridge`) to ensure the linear systems are well-posed and stable.
- **Challenge: Silent Data Corruption in Quantization:** Converting floating-point distances to `int8` or `int16` can lead to integer overflow if the distances are too large.
- *Mitigation:* I will implement strict overflow protection during the `to_quantized()` conversion, throwing explicit exceptions rather than allowing silent underflow/overflow.

Timeline

I am able to work every day of the week, with a flexible schedule to program chats and discussions with my mentors to ensure a high quality project expected to take around 175 hours.

Timeframe	Tasks & Deliverables
Until May 24	Communication with mentors, familiarization with the code and the community, the documentation and test system used. Keep learning about the API standards and mathematical algorithms to be used.
May 24 - June 25	Part-time dedication, since I will be in class and doing my final exams. I will focus on the core structural refactoring and simplest methods:
Weeks 1-2 (May 25 - June 7)	Creation of the three-class hierarchy design: <code>PairwiseMatrix</code> > <code>SymmetricMatrix</code> > <code>DistanceMatrix</code> , attributes, constructors, inheritance, simple methods and boolean flags using Python Array API.
Weeks 3-4 (June 8 - June 25)	Implementation of <code>to_square</code> and <code>to_vector</code> , attending to the logic of the attribute <code>diagonal</code> . Sparse method.
Week 5 (June 26 - July 2)	Vacations (all PRs from previous works will be submitted for review beforehand so mentors have material to evaluate during my absence.)
July 3 - August 24:	Full time dedication. I will have no more responsibilities, and will only be off for some days of vacations (not confirmed yet)
Week 6 (July 3 - July 10)	Code, documentation and revisions. Preparing Midterm Evaluation.
Weeks 7-8 (July 11 - July 26)	Using <code>LinearOperator</code> for computing partial eigen-decomposition.
Weeks 9 - 11 (July 27 - August 17)	Laplacian solves replacing “inverse” workflows.
Week 12 (August 18 - August 24)	Final modifications and tests, preparing the Final Evaluation.
Stretch Goal	If time permits, I will implement symmetrization, quantization and normalization methods.